

Harbinger com Ambiente Experimental



Eduardo Ogasawara

eogasawara@cefet-rj.br | <https://eic.cefet-rj.br/~eogasawara>



PARTE 4 - HARBINGER

Por que este bloco existe

Os capítulos anteriores dão a teoria; este bloco mostra o fluxo prático.

O objetivo não é cobrir todo o pacote, mas criar uma trilha executável para o tutorial.

A sequência vem dos exemplos em `C:\Git\da\harbinger\examples\tutorial`.

PARTE 4 - HARBINGER

A trilha prática

Começamos com o contrato comum do pacote.

Depois inspecionamos dados, visualizamos séries e rodamos detectores simples.

Por fim avaliamos resultados e conectamos anomalias, change points, suavização, SAX e motifs.

Workflow recorrente

01

Carregar dados.

02

Escolher ou configurar um método.

03

Executar detecção.

04

Inspecionar estrutura de saída.

05

Plotar e avaliar contra rótulos quando existirem.

Setup mínimo

Os exemplos usam uma semente comum para reprodutibilidade.

O pacote `harbinger` fornece detectores, avaliação e visualização.

`daltoolbox` aparece em exemplos que usam transformações e modelos auxiliares.

```
source(url("https://raw.githubusercontent.com/cefet-rj-dal/harbinger/master/examples/seed.R"))  
library(harbinger)  
library(daltoolbox)
```

Dados rotulados para o primeiro contato

O primeiro exemplo usa uma série pequena com rótulo de evento.

Isso permite entender entrada, saída e visualização sem discutir ainda um detector específico.

```
data(examples_anomalies)
dataset <- examples_anomalies$simple
head(dataset)
```

PARTE 4 - HARBINGER

Visualização inicial com rótulos

Antes de detectar, visualizamos a série e os eventos conhecidos.

Contrato comum do Harbinger

`harbinger()` representa o contrato comum do pacote.

Um método recebe uma série e retorna uma saída padronizada orientada a eventos.

Esse contrato facilita trocar detectores sem trocar todo o workflow.

```
model <- harbinger()  
detection <- detect(model, dataset$serie)  
head(detection)
```

Saída padronizada

A saída contém pelo menos índice, marcação de evento e tipo.

Detectores diferentes podem adicionar atributos específicos.

A padronização permite avaliação e plotagem com a mesma interface.

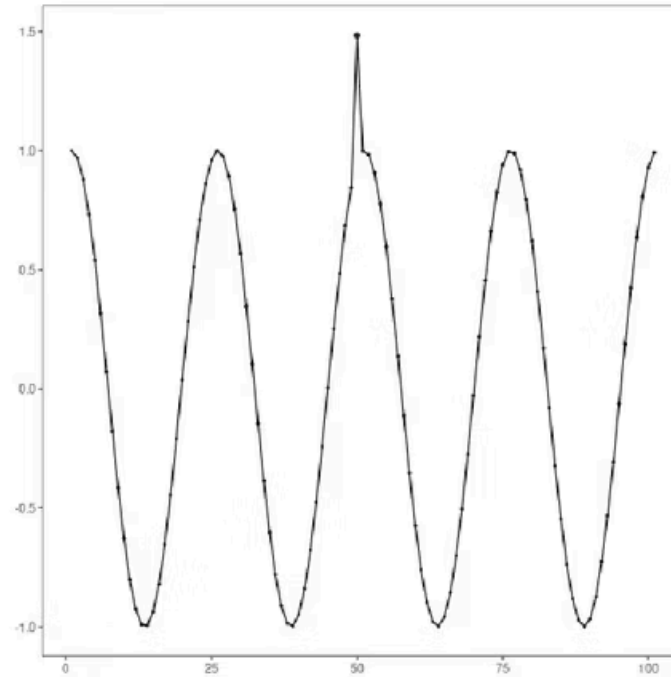
idx	event	type
-----	-------	------

1	FALSE	
---	-------	--

2	FALSE	
---	-------	--

...

Detecção e rótulos no mesmo gráfico



A função `har_plot()` combina série, detecções e ground truth.

Antes do método: conheça seus dados

- A escolha do detector depende da coleção: tamanho, canais, rótulos e estrutura.
- O tutorial inspeciona benchmarks antes de rodar métodos.
- Essa etapa evita escolher algoritmo apenas pelo nome.

Resumo de dataset

O helper identifica número de séries, tipo do dataset e colunas de sinal.

Metadados como `idx`, `event`, `type`, `seq` e `seqlen` são separados dos sinais.

Isso prepara a análise de coleções univariadas e multivariadas.

```
data(A1Benchmark)
A1Benchmark <- loadfulldata(A1Benchmark)
info <- dataset_summary(A1Benchmark)
```

PARTE 4 - HARBINGER

Benchmark Yahoo A1

Inspeção visual da primeira série de uma coleção rotulada.

PARTE 4 - HARBINGER

har_plot() como diagnóstico

Plotar não é apenas embelezar resultado.

A visualização revela tendência, bursts, mudança de nível, eventos rotulados e ruído.

É a primeira checagem antes de ajustar janelas, limiares e detectores.

Séries univariadas e multivariadas

Univariada

Uma coluna de sinal.

Entrada direta para muitos detectores.

Boa para iniciar o workflow.

Multivariada

Várias colunas de sinal.

Requer escolher canais ou combinar dimensões.

A inspeção orienta a escolha.

PARTE 4 - HARBINGER

Primeiro detector de anomalia

`hanr_histogram()` é um baseline simples e interpretável.

A ideia é tratar valores em bins raros como anômalos.

Ele não modela dinâmica temporal, mas é útil como referência inicial.

Histograma: configurar e detectar

O parâmetro `density_threshold` controla o que será considerado raro.

O detector retorna eventos do tipo `anomaly`.

```
model <- hanr_histogram(density_threshold = 0.05)
detection <- detect(model, dataset$serie)
head(detection)
```

Avaliação do baseline

Quando há ground truth, a detecção pode ser comparada com os rótulos.

O resultado inclui matriz de confusão e métricas derivadas.

```
evaluation <- evaluate(model, detection$event, dataset$event)  
evaluation$confMatrix
```

PARTE 4 - HARBINGER

Resultado do detector por histograma

Detecções do baseline sobre a série rotulada.

PARTE 4 - HARBINGER

Anomalia baseada em modelo

`hanr_arima()` modela o comportamento esperado da série.

Anomalias aparecem como resíduos grandes.

Esse exemplo conecta o Harbinger à teoria de modelos temporais.

ARIMA: ajustar, detectar e avaliar

O detector é ajustado à série antes da detecção.

O mesmo workflow `fit()`, `detect()` e `evaluate()` permanece.

```
model <- hanr_arima()  
model <- fit(model, dataset$serie)  
detection <- detect(model, dataset$serie)  
evaluate(model, detection$event, dataset$event)
```

PARTE 4 - HARBINGER

Detecção por ARIMA

O modelo detecta anomalias a partir do desvio em relação ao comportamento esperado.

PARTE 4 - HARBINGER

Resíduos e limiar

O resíduo explica por que o detector marcou determinados pontos.

Avaliação estrita e avaliação suave

Hard evaluation

Exige coincidência direta.

Útil para rótulos pontuais bem definidos.

```
har_eval()
```

Soft evaluation

Aceita desalinhamento temporal pequeno.

Mais adequada a eventos com imprecisão temporal.

```
har_eval_soft()
```

Comparando avaliadores

A mesma detecção pode ser lida por avaliadores diferentes.

Isso separa o resultado do detector do critério de julgamento.

```
hard_result <- evaluate(har_eval(), detection$event, dataset$event)  
soft_result <- evaluate(har_eval_soft(), detection$event, dataset$event)
```

Primeiro detector de change point

- `hcp_amoc()` procura uma mudança dominante.
- O alvo não é um ponto isolado raro, mas uma transição de regime.
- É uma entrada simples antes de métodos de múltiplas mudanças.

PARTE 4 - HARBINGER

Série com change point rotulado

A visualização mostra a série antes de aplicar AMOC.

PARTE 4 - HARBINGER

AMOC no Harbinger

A chamada mantém a mesma estrutura dos exemplos anteriores.

O resultado marca o índice detectado como `change point`.

```
model <- hcp_amoc()
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie, detection, dataset$event)
```

PARTE 4 - HARBINGER

Resultado com AMOC

AMOC detecta uma mudança estrutural dominante na série.

Transformações importam

- Antes de detectar, podemos transformar a série.
- Suavização revela padrões de baixa frequência e reduz variação local.
- A escolha da transformação muda o que o detector enxerga.

Moving average smoother

- `mas()` calcula média móvel.
- Janelas menores preservam detalhe.
- Janelas maiores produzem sinal mais suave.

```
ma_5 <- mas(dataset$serie, order = 5)  
ma_15 <- mas(dataset$serie, order = 15)
```

PARTE 4 - HARBINGER

MAS com janela menor

Janela menor mantém mais variação local.

PARTE 4 - HARBINGER

MAS com janela maior

Janela maior suaviza a série e evidencia movimentos mais amplos.

Representação simbólica

SAX e XSAX convertem valores numéricos em símbolos.

A representação simbólica facilita comparação de subsequências.

Ela prepara o caminho para motivos e discords.

PARTE 4 - HARBINGER

Série antes da discretização

O exemplo de motifs é transformado antes da análise simbólica.

SAX e XSAX no workflow

`trans_sax()` usa um alfabeto compacto.

`trans_xsax()` permite uma codificação mais rica.

As primeiras posições já mostram a passagem de valores para símbolos.

```
sax_model <- fit(trans_sax(alpha = 8), dataset$serie)
sax_series <- transform(sax_model, dataset$serie)
xsax_model <- fit(trans_xsax(alpha = 16), dataset$serie)
```

PARTE 4 - HARBINGER

Motifs e discords

A trilha fecha com análise de padrões recorrentes.

Motif discovery pergunta quais subsequências se repetem.

Discords são subsequências isoladas em relação ao restante.

Detector simbólico de motivos

`hmo_sax()` conecta discretização simbólica e busca por subsequências recorrentes.

Os parâmetros controlam alfabeto, tamanho da subsequência e suporte.

```
model <- hmo_sax(8, 15, 3)
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
```

PARTE 4 - HARBINGER

Resultado de motivos

Detecção orientada a motivos sobre subsequências da série.

Como usar este bloco no tutorial

- Use como ponte entre teoria e prática.
- Mostre que a mesma gramática aparece em detectores diferentes.
- Escolha poucos exemplos para executar ao vivo: primeiro detector, ARIMA, avaliação e AMOC.
- Deixe SAX/motifs como fechamento ou demonstração rápida.

PARTE 4 - HARBINGER

Mensagem principal

Harbinger organiza detecção de eventos como workflow reprodutível.

O valor está na interface comum: visualizar, detectar, avaliar e comparar.

O tutorial prático deve reforçar decisões analíticas, não apenas chamadas de função.